Практическая работа №1

Инициализация базы данных SQLite

Герасимов Кирилл Андреевич 3-ИСП44-11

Ссылка на репозиторий: https://github.com/gerasimovkirill022-jpg/prak

Кратко о работе

Цель: Создать файл базы данных SQLite и таблицу tasks с помощью DDL-команды CREATE TABLE из Python-скрипта, освоить понятия соединения, курсора и транзакции (commit).

Технологии: Python 3; встроенный модуль sqlite3; СУБД SQLite (база — один файл app.db).

Реализация: Таблица tasks содержит три колонки: id INTEGER PRIMARY KEY (заполняется автоматически), title TEXT NOT NULL (обязательное название задачи), status TEXT DEFAULT 'new' (статус по умолчанию). Благодаря IF NOT EXISTS скрипт идемпотентен: повторный запуск не вызывает ошибку и не пересоздаёт таблицу.

Основные модули:

- `src/init_db.py` — подключается к app.db (создаёт файл при первом запуске), выполняет CREATE TABLE IF NOT EXISTS tasks (id, title, status), фиксирует изменения commit() и закрывает соединение.
- `src/check_db.py` — проверочный скрипт: выводит список таблиц из системной таблицы sqlite_master и структуру таблицы tasks.

Ход работы

Шаг 1. Подготовка рабочей папки и Шаг 2. Проверка наличия модуля sqlite3

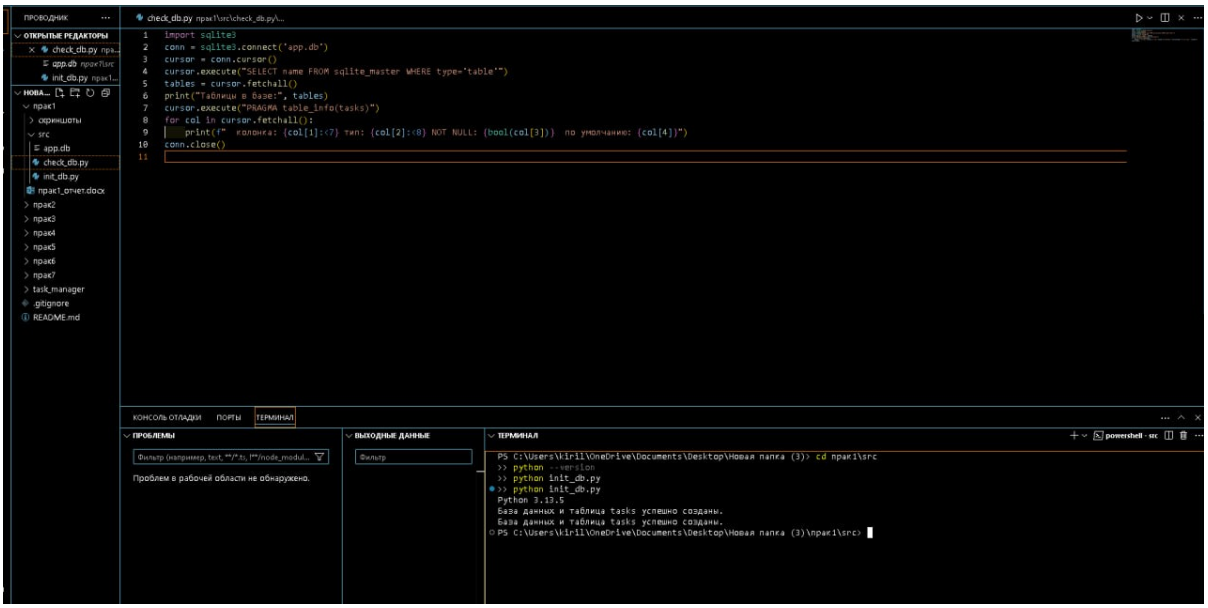


Рисунок 1 — папка проекта в редакторе и результат команды python --version

Шаг 3. Написание скрипта init_db.py

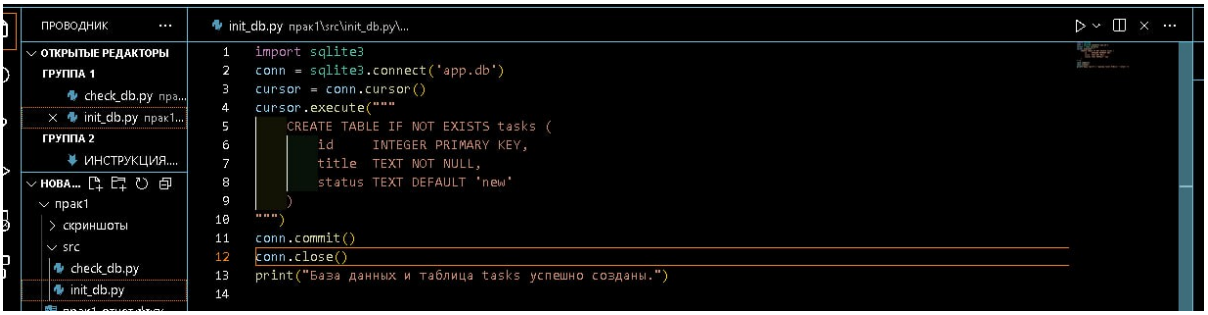


Рисунок 2 — код init_db.py в редакторе

Шаг 4. Запуск скрипта и Шаг 5. Проверка результата

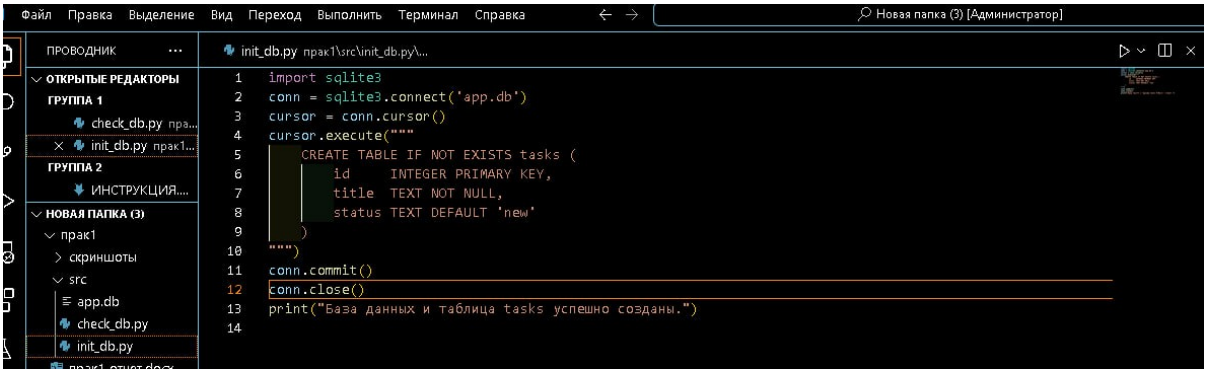


Рисунок 3 — запуск python init_db.py и появившийся файл app.db

Шаг 6. Проверка структуры базы

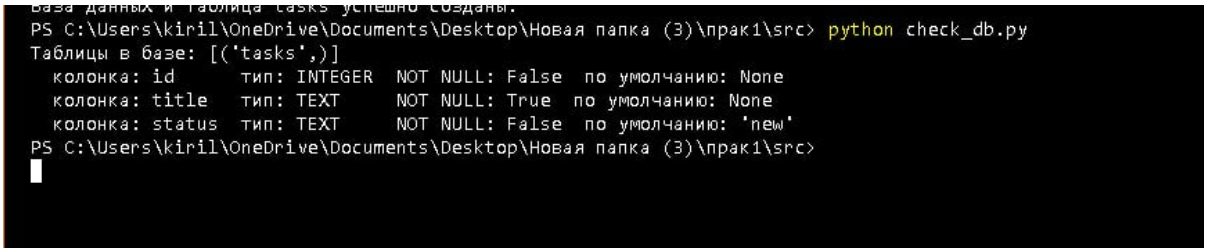


Рисунок 4 — результат python check_db.py: таблица tasks и её колонки

Шаг 7. Проверка идемпотентности

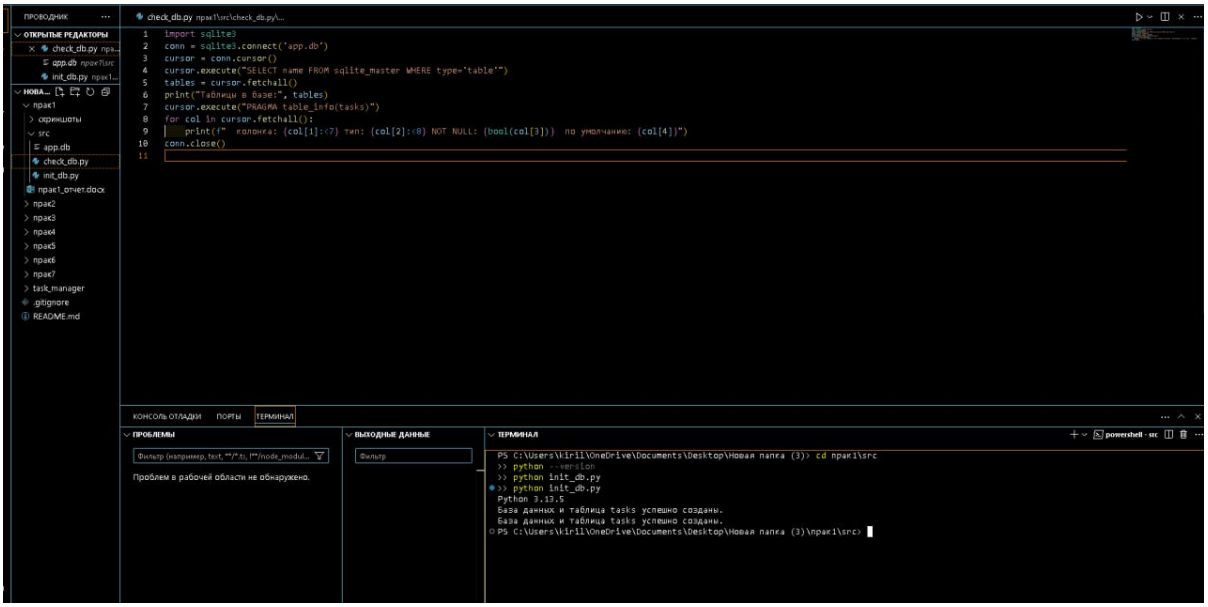


Рисунок 5 — повторный запуск init_db.py без ошибки

Примеры запросов и ответов

```
> python init_db.py
База данных и таблица tasks успешно созданы.
> python init_db.py      (повторный запуск — без ошибки)
База данных и таблица tasks успешно созданы.
> python check_db.py
Таблицы в базе: [('tasks',)]
```

Ответы на контрольные вопросы

1. **Что произойдёт, если убрать conn.commit() из скрипта и запустить его?**

Изменения останутся в незавершённой транзакции и не будут гарантированно записаны в файл. При закрытии соединения без commit() незафиксированные изменения откатываются. (В Python 3.6–3.11 модуль sqlite3 выполняет CREATE TABLE вне неявной транзакции, поэтому таблица может появиться и без commit, но полагаться на это нельзя — commit() обязателен по правилам работы с транзакциями.)

2. **Чем отличается connection от cursor? Зачем нужны оба?**

Connection — канал связи с файлом базы; через него управляют транзакцией (commit, rollback) и закрывают доступ (close). Cursor создаётся из соединения и выполняет конкретные SQL-запросы (execute) и забирает результаты (fetchone, fetchall). Одно соединение может иметь несколько курсоров.

3. **Почему id INTEGER PRIMARY KEY заполняется автоматически?**

В SQLite колонка INTEGER PRIMARY KEY является псевдонимом встроенного rowid. Если значение не передано, СУБД сама присваивает следующее целое число (обычно максимальный id + 1).

4. **Что делает DEFAULT 'new' в колонке status?**

Задаёт значение по умолчанию: если при INSERT статус не указан, в колонку автоматически записывается 'new'.

5. **Где физически на диске лежат данные таблицы tasks?**

В файле app.db в папке, из которой был запущен скрипт. SQLite хранит всю базу (все таблицы и данные) в одном файле.

Вывод

В ходе работы создана база данных SQLite и таблица tasks, изучены DDL-команды, роль соединения и курсора, необходимость commit() и close(). Проверено, что скрипт инициализации идемпотентен.